RouteMate AI

A beginner's guide to what we built, how, and how to run it

Codegnan Python Group Project · Batch PFS-66

Who this guide is for

You know basic Python (variables, lists, dictionaries, functions, loops) and want to understand our project well enough to run it, explain it in the viva, and change it. You do *not* need to know graph theory, Flask or AI beforehand; every idea is introduced from scratch.

Contents

1 What is RouteMate AI? 2

2 Tools and technologies 2

3 How to run the application 3

3.1 One-time setup . . . 3

3.2 Start the website . . . 3

3.3 Run the tests . . . 4

3.4 If something goes wrong . . . 4

4 A tour of the folders 4

5 How it works, step by step 5

5.1 The metro is a graph . . . 5

5.2 Why one station becomes several nodes . . . 6

5.3 Trains do not wait for you: the timetable . . . 6

5.4 All the algorithms, in plain words . . . 6

5.5 Data structures we built . . . 6

5.6 The AI part . . . 7

6 Following one question through the code 7

7 Python concepts you will meet in the code 8

8 How we know it works: the tests 8

9 Glossary 8

10 Get familiar: things to try 9

1 What is RouteMate AI?

RouteMate AI is a website that plans trips on the **Hyderabad Metro**. You type a question in normal English (or Telugu or Hindi), for example:

> *"I'm at Ameerpet, need to reach Hitech City before 10, don't want to change trains much"*

and the app answers with:

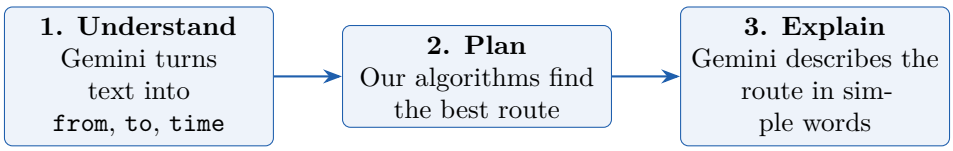
- which trains to take, where to change, and at what time to leave;
- how long the trip takes, how many stops, and the fare in rupees;
- the route drawn on a map, plus a short friendly explanation.

The network in the app is real: **3 lines** (Red, Blue, Green) and **57 stations**, with the official fare chart (Rs. 11 to Rs. 69).

The one idea to remember

The AI never picks the route

The AI (Google Gemini) does only two jobs: it **reads** your question and it **explains** the answer. The route itself is always computed by *our own Python code* using classic algorithms. So the answer is always a real, correct route, never a guess.



no AI in step 2 — pure Python

If there is no Gemini key (or the internet is down), steps 1 and 3 fall back to simple built-in Python rules, so the app *still works*; it just sounds less chatty.

2 Tools and technologies

Tool	What it is and why we use it
Python 3.12	The language for all the backend logic.
uv	A fast tool that installs Python packages and runs our app. It replaces `pip` + `venv`: one command (`uv sync`) sets everything up.
Flask	A small web framework. It lets a Python function answer a web address, e.g. `/api/route`.
SQLite	A database stored in a single file (`var/routemate.db`). Holds stations, lines, fares and a log of searches. Comes built into Python.
Google Gemini	The AI model (`gemini-3.1-flash-lite`), used through Google's `google-genai` package.
python-dotenv	Reads secret settings (like the API key) from a `.env` file.

pytest	Runs our 117 automatic tests.
HTML, CSS, JavaScript	The web page you see in the browser.
Leaflet + OpenStreetMap	A free JavaScript map library and free map tiles, used to draw the metro lines and your route.
Git + GitHub	Version control; the code lives in a private GitHub repository.

Everything else — the graph, the priority queue, the autocomplete tree, the typo matcher and every search algorithm — we **wrote ourselves** in plain Python. That is the "heavy lifting" of the project.

3 How to run the application

3.1 One-time setup

Step 1 — Install uv. On macOS or Linux, open a terminal and run:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

On Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close and reopen the terminal, then check it worked with `uv --version`. You do not need to install Python separately; uv downloads the right version for you.

Step 2 — Get the code. The repository is private, so ask the team leader to add your GitHub account as a collaborator first. Then:

```
git clone https://github.com/nithin-raj-9100/routemate-ai.git
cd routemate-ai
```

Step 3 — Install the dependencies.

```
uv sync
```

This creates a hidden `.venv` folder containing Flask, google-genai and the rest.

Step 4 — (Optional) Add your Gemini API key. Get a free key from https://aistudio.google.com/apikey. Then copy the example settings file:

```
cp .env.example .env        # Windows: copy .env.example .env
```

Open `.env` in any editor and paste your key after `GEMINI_API_KEY=`.

Keep the key secret

`.env` is listed in `.gitignore`, so Git will never upload it. Never paste your key into code, screenshots or chat groups.

Step 5 — Create the database.

```
uv run routemate init-db
```

3.2 Start the website

```
uv run routemate serve
```

Now open http://127.0.0.1:5050 in your browser. You will see three tabs:

- **Ask** — type a question like a chat. Try the example chips.
- **Plan** — a normal form: From, To, "Optimise for", and a time. You can also click stations on the map.
- **Algorithms** — runs every algorithm on the same trip and shows how much work each one did.

Press `Ctrl+C` in the terminal to stop the server.

3.3 Run the tests

```
uv run pytest
```

You should see `117 passed`. Run this after every change you make.

3.4 If something goes wrong

Problem	Fix
`uv: command not found`	Reopen the terminal after installing uv.
The page says "AI offline"	No valid key in `.env`. The app still works using the offline rules.
Port 5050 already in use	Another copy is running; stop it, or run `uv run routemate serve --port 5051`.
The key is ignored	Run the commands from inside the `routemate-ai` folder; that is where `.env` is read from.

4 A tour of the folders

All the Python code is inside src/routemate/. Each folder has one job:

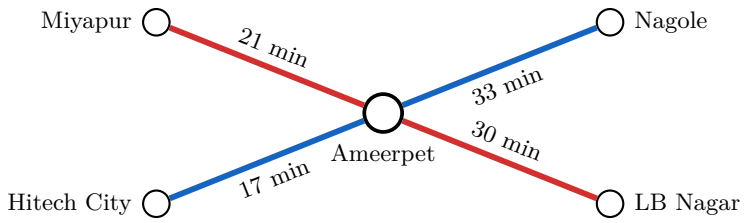
Folder / file	What lives there
data/hyderabad_metro.json	The raw data: stations, map coordinates, nicknames ("MGBS", "Koti"), lines, fares, train frequency.
ds/	**D**ata **s**tructures we wrote ourselves: heap, trie, graph, typo matching.
algorithms/	Search algorithms: Dijkstra, A*, BFS, 0-1 BFS, Yen, Pareto.
network/	Turns the metro data into a graph the algorithms can search.
planner/	The "brain": timetable, route planning, building the step-by-step itinerary.
nlp/	Language and AI: station name matching, the rule parser, Gemini parser and explainer, and the `Assistant` that ties them together.

web/	The Flask server (api.py), the HTML page, the JavaScript and the CSS.
db.py	Creates and fills the SQLite database.
config.py	Reads settings from .env.
services.py	Builds all the objects once when the app starts.
cli.py	The two commands: init-db and serve.
tests/	117 automatic tests.

5 How it works, step by step

5.1 The metro is a graph

A **graph** is just *dots* (called *nodes*) joined by *lines* (called *edges*). Stations are the dots, tracks are the edges, and each edge has a *cost*: how many minutes it takes.



A tiny piece of the network. Ameerpet is an *interchange*: two lines meet there. (Times simplified.)

"Find the fastest route" now becomes a well-known computer science problem: *find the cheapest path between two nodes*. The classic answer is **Dijkstra's algorithm**. Here is a complete, beginner-sized version you can run:

```
import heapq

graph = {
    "Miyapur":     [("Ameerpet", 21)],
    "Hitech City": [("Ameerpet", 17)],
    "Ameerpet":    [("Miyapur", 21), ("Hitech City", 17), ("LB Nagar", 30)],
    "LB Nagar":    [("Ameerpet", 30)],
}

def dijkstra(graph, start, goal):
    queue = [(0, start, [start])]      # (minutes so far, station, path)
    visited = set()
    while queue:
        minutes, station, path = heapq.heappop(queue)   # always the closest one
        if station == goal:
            return minutes, path
        if station in visited:
            continue
        visited.add(station)
        for next_station, cost in graph[station]:
            heapq.heappush(queue, (minutes + cost, next_station, path + [next_station]))
    return None

print(dijkstra(graph, "Miyapur", "Hitech City"))
# (38, ['Miyapur', 'Ameerpet', 'Hitech City'])
```

The trick is the **priority queue** (`heapq`): it always hands back the station we can reach soonest, so the first time we pop the goal, we know no faster way exists. Our real version in `algorithms/shortest.py` is the same idea, with our own heap (`ds/heap.py`) instead of `heapq`.

5.2 Why one station becomes several nodes

The toy graph above has a problem: it cannot tell *staying on the train* at Ameerpet from *getting off and changing lines*, which takes about 3 minutes of walking. So in the real graph (`network/transit_graph.py`) each station is split into several nodes:

- **waiting** on the platform of one line (e.g. "Ameerpet, Blue platform"), and
- **riding** a train of one line in one direction (e.g. "Ameerpet, on a Red train towards LB Nagar").

Edges then have *kinds*: **board** (platform → train), **ride** (station → next station), **transfer** (walk to another line's platform, counts as one change) and **alight** (get off). This is called a *line-expanded graph*, and it lets us count line changes exactly.

5.3 Trains do not wait for you: the timetable

Trains leave every 4 minutes at peak hours and every 6–8 minutes otherwise, from 06:00 to 23:00 (`planner/timetable.py`). So the cost of a *board* edge depends on the clock: arrive at 08:43 and the next train is at 08:44, so you wait 1 minute. Because the cost of an edge depends on the time you reach it, this is called **time-dependent** Dijkstra.

For "arrive by 10:00" we use **binary search**: try a departure time halfway through the range, see if it arrives in time, and halve the range again, until we find the *latest* time you can leave.

5.4 All the algorithms, in plain words

Algorithm	Question it answers
Dijkstra	What is the fastest route?
A* ("A-star")	The same answer, but faster to compute: it uses the straight-line distance to the destination as a hint, so it looks in the right direction first.
BFS	What is the route with the fewest stations? (Every edge counts as 1.)
0-1 BFS	What is the route with the fewest line changes? (Edges cost 0 or 1.)
Yen's algorithm	What are the 2nd and 3rd best routes, which are genuinely different?
Pareto search	Show every sensible trade-off, e.g. "fastest with 2 changes" vs "5 minutes slower but no change".
Binary search	What is the latest time I can leave and still arrive by a deadline?

The **Algorithms** tab of the website runs them side by side and shows how many nodes each one looked at. A* usually looks at far fewer nodes than Dijkstra, for the same answer.

5.5 Data structures we built

- **Heap** (`ds/heap.py`) — the priority queue that Dijkstra and A* use.
- **Trie** (`ds/trie.py`) — a tree of letters for *autocomplete*. Type `ame` and it walks A→M→E to find "Ameerpet" instantly, without checking all 57 names.

- **Edit distance** (ds/fuzzy.py) — counts how many letter changes turn one word into another. "Ameerpeth" is 1 change away from "Ameerpet", so typos still work.
- **Graph** (ds/graph.py) — a dictionary mapping each node to its list of edges.

5.6 The AI part

Understanding the question (nlp/llm_parser.py). We send Gemini the question together with the list of real stations, and ask it to reply in strict JSON:

```
{"origin": "Ameerpet", "destination": "Hitech City",
 "mode": "fewest_transfers", "arrive_by": "10:00", "avoid": [], "language": "en"}
```

Gemini is good at landmarks ("Inorbit Mall" → Durgam Cheruvu station) and at Telugu or Hindi. But we **never trust it blindly**: every station name it returns is checked against our station list, and unknown ones are dropped.

The offline rule parser (nlp/rules.py) always runs too. It uses plain Python and regular expressions: words like *from* or *at* mark the start, *to* or *reach* mark the destination, and patterns like "before 10" or "by 9:30 am" become times. If Gemini is missing or leaves a field empty, these rules fill the gap.

Explaining the answer (nlp/explain.py). After the planner finds the route, we give Gemini only the verified facts (lines, stations, times, fare) and ask for a short, friendly explanation, with the instruction not to invent anything. Without Gemini, a fill-in-the-blanks template writes the explanation instead.

Safety switch. If the API key is wrong, or Gemini fails 3 times in a row, the app stops calling it for a while (a *circuit breaker*) instead of making you wait.

Streaming. Like ChatGPT, the answer appears word by word. The server sends small pieces as soon as they are ready, using *Server-Sent Events*, and the page types them out.

6 Following one question through the code

What happens when you type *"cheapest way from LB Nagar to Miyapur"* and press Ask:

1. The browser (web/static/app.js) sends the text to `POST /api/ask/stream`.
2. Flask (web/api.py) receives it and calls `Assistant.ask_stream` in nlp/assistant.py.
3. The assistant *understands* the text: Gemini and the rule parser each produce a `TripQuery` (from, to, mode, time), and the two are combined.
4. Station names are matched to real stations by nlp/stations.py (trie + typo matching).
5. `RoutePlanner.plan` (planner/planner.py) runs A* on the transit graph, using the timetable for waiting times.
6. The result is turned into readable steps, and the fare is looked up (planner/itinerary.py).
7. The explainer streams a friendly explanation back, piece by piece.
8. The page shows the steps, types out the explanation and draws the route on the map.
9. The search is saved in the SQLite `search_log` table.

7 Python concepts you will meet in the code

Concept	What it means	Example file
Packages and modules	A folder with `__init__.py` is a package; each `.py` file is a module.	`src/routemate/`
Classes	Group data and the functions that use it.	`planner/planner.py`
`@dataclass`	A short way to write classes that mainly hold data.	`ds/graph.py`
Type hints	`def f(x: int) -> str` documents what goes in and out.	`everywhere`
Generators (`yield`)	A function that hands back results one at a time; used for streaming.	`nlp/assistant.py`
Decorators	`@api.get("/health")` tells Flask which URL a function answers.	`web/api.py`
`sqlite3`	Python's built-in database module.	`db.py`
`json`	Read and write JSON data.	`db.py`
Exceptions	`raise PlanningError(...)` reports problems like "no train at this hour".	`planner/planner.py`
pytest	Functions named `test_...` with `assert` lines.	`tests/`

8 How we know it works: the tests

The `tests/` folder has 117 tests that run in about 3 seconds. The most interesting ones check our algorithms against **brute force**: on random graphs, a slow but obviously correct method (like trying every possible path) must give exactly the same answer as our fast algorithm. Other tests check the planner ("step times add up to the total"), the AI layer (using a *fake* Gemini so no internet is needed) and the website (through Flask's test client).

9 Glossary

Node / edge A dot and a connecting line in a graph.

Interchange A station where lines meet: Ameerpet, MG Bus Station, Parade Ground.

Headway Minutes between two trains on the same line.

Heuristic A smart guess that guides a search (A* uses straight-line distance).

API A web address that returns data (JSON) instead of a web page, e.g. `/api/route`.

API key A secret password that lets our app use Gemini.

JSON A text format for data that looks like Python dictionaries and lists.

LLM Large Language Model, the kind of AI that Gemini is.

Fallback What we do when the preferred option fails (rules instead of Gemini).

10 Get familiar: things to try

1. Ask: *Uppal to Jubilee Hills Check Post at 6pm, avoid Ameerpet.* Then ask the follow-up *what if I want the fastest?*
2. In the Plan tab, pick "Arrive by" 10:00 and see the latest time you can leave.
3. Open the Algorithms tab and compare how many nodes Dijkstra and A* settle.
4. Visit `http://127.0.0.1:5050/api/route?from=Ameerpet&to=Uppal` to see the raw JSON.
5. Misspell a station ("dilsuknagar") and watch it still work.
6. Change a headway in `data/hyderabad_metro.json`, restart the server and see the waiting times change.
7. Break something on purpose, run `uv run pytest`, and see which test catches it.